

XBOT EV - TASK 4

Employer-Focused Questions & Answers

Purpose -> Tools -> Build Process -> Business Value -> Contribution -> Testing -> Learning

Interview focus: explain what you built, why you built it, which tools you used, how you created it, and how it benefits XBOT EV. AI tools are development assistants, not runtime dependencies.

1. What was the purpose of Task 4?

The purpose of Task 4 was to build a working lead-response automation for XBOT EV. When a potential customer submits their name, email, and interest, the system automatically sends a personalized acknowledgement and records the lead for follow-up. From a business perspective, the purpose is to reduce manual first-response work, make sure customer enquiries are not lost, and create a more professional and traceable lead-handling process.

2. What exactly did you build in Task 4?

I built a production-deployed full-stack lead-response system. The customer submits a lead through the XBOT EV interface. The backend validates and stores the lead in Supabase, then securely triggers an n8n automation. n8n prepares a personalized response using the customer's name and stated interest, sends the email through Resend, and reports the result back to the application. The system also tracks whether the lead is pending, processing, sent, or failed. So the project is not just a form that sends an email; it is a complete lead-capture and automation workflow with persistence, security, and traceability.

3. How many main tools and technologies did you use?

Using a strict count, I used 14 main tools and technologies. I separate the production/development stack from the AI-assisted development tools so the role of each one is clear.

4. Which tools did you use?

I used the following 14 tools and technologies, with a clear purpose for each:

Tool	Purpose
1. Next.js	Full-stack application and trusted server APIs.
2. React	Interactive customer-facing lead form and UI behaviour.
3. TypeScript	Safer, more maintainable application logic and data contracts.
4. Tailwind CSS	Responsive XBOT EV interface and styling.
5. Zod	Authoritative server-side validation.
6. Supabase PostgreSQL	Persistent lead storage and lifecycle tracking.
7. n8n Cloud	Workflow automation and orchestration.
8. Resend	Transactional acknowledgement email delivery.
9. Vercel	Production deployment and environment configuration.

10. Git	Version control and change history.
11. GitHub	Repository hosting, handover, and deployment integration.
12. ChatGPT	Primary AI-assisted planning, debugging, review, and documentation support.
13. Claude	Secondary AI-assisted technical review and comparison of approaches.
14. Gemini	Additional AI-assisted research, cross-checking, and alternative suggestions.

Important distinction: Next.js through GitHub form the application/development stack. ChatGPT, Claude, and Gemini were AI-assisted development tools and are not production runtime dependencies.

5. Why did you choose these tools?

I wanted the smallest architecture that could still demonstrate a real production-style workflow. Next.js gave me frontend and trusted backend APIs in one deployment. Supabase gave me durable PostgreSQL storage. n8n made the automation visible and demonstrable. Resend provided a transactional email API. Vercel provided simple public deployment. Git and GitHub supported source control and handover. ChatGPT, Claude, and Gemini accelerated planning, review, debugging, and cross-checking without becoming runtime dependencies.

6. How did you create Task 4 from start to finish?

I started with the MVP requirement: lead form -> automation -> personalized email. Then I added a trusted backend so the browser would not hold privileged secrets. Next, I added Supabase persistence so every valid enquiry would be stored before automation started. I connected the backend to an authenticated n8n webhook, configured n8n to prepare the customer-specific email and send it through Resend, and added a secure callback so the application could record processing and sent states. Finally, I tested duplicates, failure conditions, security checks, production deployment, and documented the architecture and case study.

7. What is the complete technical flow of Task 4?

The flow is: Customer -> React form -> Next.js API -> Zod validation -> Supabase -> authenticated n8n webhook -> Resend -> authenticated callback -> Supabase. The key reliability decision is that the lead is stored before external automation is triggered.

8. Why did you store the customer lead in a database?

Because a customer lead is business data, and business data should not depend only on a browser request or an external automation platform. Supabase stores the lead independently of the browser and keeps lifecycle status, workflow IDs, provider message IDs, timestamps, and diagnostics. This gives XBOT EV a traceable record of what happened after each enquiry.

9. Why did you use n8n?

I used n8n because Task 4 specifically required a working automation. n8n makes the workflow visible and inspectable. It receives the lead from the application, records that processing has started, prepares the personalized email data, calls Resend, and reports the result back to the application. It

also makes the automation easier for an employer to inspect than if everything were hidden inside backend code.

10. Why did you use Resend?

I used Resend as the transactional email provider. The application should not attempt to build its own mail-delivery infrastructure. Resend provides a clean API for sending the acknowledgement and returns a provider operation ID that can be stored for traceability. That gives the system stronger evidence than simply assuming the message was sent.

11. Why did you choose email instead of WhatsApp?

The assessment allowed either email or WhatsApp. I chose email because it gave me a simpler and more reproducible implementation while still fully meeting the requirement. It also made provider testing, inbox verification, and traceability easier for the assessment. If required later, the same architecture could be extended with a WhatsApp provider without changing the core persistence model.

12. What business value does Task 4 provide to XBOT EV?

The main value is faster and more consistent lead acknowledgement. A customer does not have to wait for someone to manually respond to the initial enquiry. At the same time, the company retains the enquiry in a database and can trace whether automation started and whether the email provider accepted the response. This can reduce repetitive first-response work and make lead handling more organized.

13. Which teams could benefit from this system?

Sales can focus on qualified follow-up instead of manually acknowledging every enquiry. Marketing can retain campaign-generated leads in a traceable workflow. Customer-service teams can see whether a response progressed successfully. Operations and management can use lifecycle information and diagnostics to investigate issues. The same pattern could later support test rides, dealer enquiries, service requests, bookings, or fleet enquiries.

14. What did you personally contribute to Task 4?

I worked hands-on across the architecture and integrations. I configured the frontend and backend flow, Supabase persistence, n8n workflow, Resend integration, Vercel deployment, security configuration, duplicate protection, lifecycle logic, production testing, and documentation. I also used AI-assisted development where useful, while validating the final behaviour through actual system evidence.

15. Which AI tools did you use while completing Task 4?

I used ChatGPT, Claude, and Gemini as AI-assisted development tools. ChatGPT was my primary assistant for implementation planning, architecture discussions, debugging support, workflow review, documentation refinement, and quality checks. Claude was used as a secondary technical reviewer for comparing approaches and reviewing code or design decisions. Gemini was used for additional research, cross-checking, and alternative technical suggestions. I did not treat AI output as automatically correct; I verified the final implementation through the live application, Supabase records, n8n executions, API behaviour, Resend provider responses, and actual email testing.

16. Did you use an AI model or LLM inside the actual production automation?

No. ChatGPT, Claude, and Gemini supported the development process, but none of them are runtime dependencies in the production automation. The workflow already knows the customer's name and stated interest, so a deterministic message template is simpler, safer, faster, cheaper, and easier to test. Adding an LLM would introduce unnecessary latency, unpredictability, cost, and hallucination risk.

17. How did you make Task 4 secure?

I kept privileged secrets on trusted server-side boundaries. The browser does not directly call n8n using private credentials. The Next.js server authenticates when triggering n8n, and n8n uses separate authentication when calling the application callback. The Resend key is stored in n8n credential management, while production application secrets are stored in Vercel environment variables. Callback payloads and lifecycle transitions are validated rather than relying only on authentication.

18. How did you make the system reliable?

One of the main principles was: persist before automate. The lead is stored before n8n is triggered. I also implemented lifecycle states, duplicate protection, authenticated callbacks, safe failure handling, and traceability information. This means the system does not simply say "form submitted"; it records what actually happened downstream.

19. How did you prevent duplicate emails or duplicate leads?

Each logical submission receives a UUID called a submission ID. The database enforces uniqueness for that identifier. If the same request is received again, the API returns the existing lead instead of creating another lead and triggering automation again. This protects against retries, repeated API calls, and some race-condition scenarios.

20. What was the most difficult problem you faced?

One of the strongest technical problems occurred inside n8n. After an HTTP Request node executed, the current JSON context changed. Later workflow nodes could no longer safely assume that the customer information was still available in the same \$json location. I diagnosed the data-lineage issue and fixed it by explicitly referencing the original Webhook node. That taught me that visual automation platforms still require careful reasoning about state and data flow.

21. How did you solve problems during development?

I followed a structured debugging approach: symptom -> reproduce -> gather evidence -> identify root cause -> minimal fix -> regression test. For example, when production automation did not work correctly, I inspected the database record and diagnostic information. That showed persistence had succeeded but the production n8n webhook configuration was missing. I corrected the environment configuration, redeployed, and tested the complete flow again.

22. How did you prove Task 4 actually worked?

I tested the complete production chain. I verified real form submission, server-side validation, Supabase persistence, n8n execution, the processing callback, Resend acceptance, the final callback, final database status, and actual email receipt. I also tested duplicate requests, unauthorized

callbacks, malformed identifiers, invalid state transitions, production build, linting, and responsive UI behaviour.

23. What makes your Task 4 solution stronger than a basic assessment submission?

A minimal submission could have been form -> automation -> email. I kept that requirement as the MVP but strengthened the system with server-side persistence, validation, authenticated integrations, lifecycle tracking, duplicate protection, failure semantics, traceability, and production deployment. The goal was not to make the architecture unnecessarily complex; it was to show that I can take a small automation requirement and think about how it should behave as a real system.

24. What are the limitations of your current solution?

The current version does not include CRM integration, WhatsApp, a durable queue, transactional outbox, full email-delivery webhook tracking, or production-verified handling for every possible provider failure. It also does not guarantee exactly-once distributed execution or guaranteed inbox delivery. I prefer to state these limitations clearly rather than overclaim what the current system proves.

25. What would you improve if XBOT EV wanted to use this at larger scale?

I would add stronger observability, automated integration testing, provider delivery events, CRM integration, a transactional outbox or durable queue, stronger retry controls, rate controls, and formal database migrations. For a larger operation, I would also add dashboards and alerts around failed or uncertain lead states. The current solution demonstrates the architecture and workflow principles, while those additions would improve resilience and operations at greater scale.

Strong Final Interview Statement

Task 4 gave me hands-on exposure to the full lifecycle of a real automation system: understanding the business requirement, selecting the right tools, building the frontend and backend flow, persisting business data, integrating external services through APIs, securing trust boundaries, debugging production issues, testing the real workflow, and documenting limitations honestly. The most important lesson for me was that successful engineering is not about using the largest number of tools. It is about choosing the right tools, understanding why each one exists, and proving that the complete system works.

Quick Memory Flow

- Requirement: lead form -> automatic personalized reply.
- Business purpose: capture every valid enquiry and acknowledge it quickly.
- Architecture: Next.js -> Zod -> Supabase -> n8n -> Resend -> callback -> Supabase.
- Reliability: persist before automate.
- Security: secrets remain server-side and integrations are authenticated.
- AI assistance: ChatGPT + Claude + Gemini during development, not runtime.
- Testing: verify the complete production chain, not only the frontend.
- Engineering judgment: use the smallest secure, testable, reliable solution.